Complete programs for incidence representations

Human Compression Project

This appendix contains every declared mathematical program: one required finite graph certificate and nine optional validation files. The complete `certificate/graph.json` adjacency and generator data is also mandatory for the required certificate; it is supplied as a whole machine-readable file, not replaced by a digest or by this appendix. The uniform theorems have written proofs; the bounded runs do not establish them. The orchestration driver and scoped Lean files are inventoried separately. No executable listing is hidden inside either scientific application.

panel_permutations.py

```
#!/usr/bin/env python3
"""Bounded diagnostics for the new infinite panel-permutation proof.

The written syllable argument is the proof. This file tests actual reduced
words and all chosen left cyclic moves; it is not an infinite enumeration.
"""
import argparse,itertools,json
from pathlib import Path

def need(ok,message):
    if not ok: raise ValueError('L126_PANEL_REJECT '+message)

def reduce(w):
    a=[]
    for x in w:
        if a and a[-1]==-x:a.pop()
        else:a.append(x)
    return tuple(a)

def mul(a,b):return reduce(a+b)
def inv(a):return tuple(-x for x in reversed(a))
def power(i,n):return (i if n>0 else -i,)*abs(n)
def words(r,length):
    a=[()];yield ()
    for _ in range(length):
        a=[w+(x,) for w in a for x in range(-r,r+1) if x and (not w or w[-1]!=-x)]
        yield from a

def act(w,i,p,first=False):
    if not w:return w
    if first:
        if abs(w[0])!=i:return w
        end=1
        while end<len(w) and w[end]==w[0]:end+=1
        n=end*(1 if w[0]>0 else -1)
        return power(i,p.get(n,n))+w[end:]
    if abs(w[-1])!=i:return w
    start=len(w)-1
    while start and w[start-1]==w[-1]:start-=1
    n=(len(w)-start)*(1 if w[-1]>0 else -1)
    return w[:start]+power(i,p.get(n,n))

def check(first=False):
    r=3;i=1;p={1:2,2:1};q={-1:1,1:-1}
    support=set(p)|set(q)
    pq={n:p.get(q.get(n,n),q.get(n,n)) for n in support}
    total=0;panels=0
    for w in words(r,5):
```

```
        z=act(w,i,p,first)
        need(reduce(z)==z,'reduced image')
        need(act(z,i,p,first)==w,'inverse')
        need(act(act(w,i,q,first),i,p,first)==act(w,i,pq,first),'composition')
        for j in range(1,r+1):
            for n in [-2,-1,1,2]:
                h=mul(power(j,n),w)
                image_h=act(h,i,p,first)
                difference=mul(image_h,inv(z))
                need(all(abs(x)==j for x in difference),'labeled panel adjacency')
                panels+=1
        total+=1
    # The type i cosets detect the nontrivial symmetric permutation.
    need(act((i,),i,p)==(i,i),'faithfulness witness')
    return dict(status='PASS_BOUNDED_PANEL_DIAGNOSTICS',rank=r,max_word_length=5,
                reduced_words=total,panel_pairs=panels,uniform_proof=False)

def main():
    p=argparse.ArgumentParser();p.add_argument('--out',type=Path,required=True)
    p.add_argument('--alter-first',action='store_true');a=p.parse_args()
    result=check(a.alter_first)
    need(not a.out.exists(),'fresh output')
    a.out.write_text(json.dumps(result,indent=2)+'\n')
if __name__=='__main__':main()
```

phase177_hexagon_check.g

```
LoadPackage("grape");;
SetAssertionLevel(1);;

P177Hexagon := function(tips, expected)
  local n, edges, j, i, gr, colors, A, C, gen, cls, reflection;
  n:=6+Length(tips);
  edges:=List([0..5],i->Set([i+1,((i+1) mod 6)+1]));
  for j in [1..Length(tips)] do
    i:=tips[j];
    Add(edges,Set([i+1,6+j]));
    Add(edges,Set([((i+1) mod 6)+1,6+j]));
  od;
  gr:=Graph(Group(()),[1..n],OnPoints,
    function(a,b) return Set([a,b]) in edges; end,true);
  colors:=[[1,3,5],[2,4,6],[7..n]];
  A:=AutGroupGraph(gr,colors);
  C:=AutGroupGraph(gr);
  for gen in GeneratorsOfGroup(C) do
    for cls in colors do Assert(1,OnSets(cls,gen) in colors); od;
  od;
  Assert(1,[Size(C),Size(A)]=expected);
  Print("HEXAGON tips=",tips," correlation_order=",Size(C),
    " type_preserving_order=",Size(A)," groups=",
    StructureDescription(C),"/",StructureDescription(A),"\n");
  if Length(tips)=6 then
    reflection:=PermList(Concatenation(
      List([0..5],i->(-i mod 6)+1),
      List([0..5],i->((-i-1) mod 6)+7)));
    Assert(1,reflection in A and Order(reflection)=2);
    Print("TYPE_PRESERVING_REFLECTION ",reflection,"\n");
  fi;
end;;

P177Hexagon([0,2,4],[6,3]);;
P177Hexagon([0..5],[12,6]);;
Print("PHASE177_HEXAGON_CHECK_COMPLETE\n");
QUIT;
```

phase188_gadget_check.g

```
# Reviewer-written bounded checks of the three F010 constructions.
# Universal claims depend on the reviewed written proofs, not these examples.
LoadPackage("grape");;
```

```
SizeScreen([100000,24]);;
R188Check:=function(b,s) if not b then Error(Concatenation("phase188: ",s));fi;end;;
R188Data:=function(colors,edges)
 return rec(colors:=colors,edges:=Set(List(edges,Set)),n:=Length(colors));
end;;
R188Graph:=function(d)
 return Graph(Group(()),[1..d.n],OnPoints,function(a,b)return Set([a,b]) in
  ↪  d.edges;end,true);
end;;
R188Pair:=function(d)
 local palette,classes,n,r,aug,A,C,K;
 n:=d.n;palette:=Set(d.colors);r:=Length(palette);
 classes:=List(palette,c->Filtered([1..n],i->d.colors[i]=c));
 aug:=Graph(Group(()),[1..n+r],OnPoints,function(a,b)
  if a<=n and b<=n then return Set([a,b]) in d.edges;fi;
  if a>n and b>n then return false;fi;
  if a<=n then return d.colors[a]=palette[b-n];fi;
  return d.colors[b]=palette[a-n];
 end,true);
 A:=AutGroupGraph(aug,[[1..n],[n+1..n+r]]);
 C:=Action(A,[1..n],OnPoints);
 R188Check(Size(C)=Size(A),"palette encoding not faithful");
 K:=AutGroupGraph(R188Graph(d),classes);
 R188Check(IsSubgroup(C,K) and IsNormal(C,K),"kernel normality");
 return [C,K];
end;;
R188Degrees:=function(d)
 return List([1..d.n],v->Number(d.edges,e->v in e));
end;;
R188TriangleFree:=function(d)
 return ForAll(d.edges,e->not ForAny([1..d.n],v->Set([v,e[1]]) in d.edges and Set([v,e[2]])
  ↪  in d.edges));
end;;
R188Connected:=function(d)
 local reached,old;
 reached:=[1];old:=[];
 while reached<>old do old:=reached;reached:=Union(old,Concatenation(Filtered(d.edges,e->In
  ↪  tersection(e,old)<>[])));od;
 return Length(reached)=d.n;
end;;
R188CompareRestriction:=function(old,new)
 local p,q,restriction;
 p:=R188Pair(old);q:=R188Pair(new);
 for restriction in [1,2] do
  R188Check(ForAll(GeneratorsOfGroup(q[restriction]),g->OnSets([1..old.n],g)=[1..old.n]),"o
   ↪  ld vertices not invariant");
  R188Check(Size(q[restriction])=Size(p[restriction]) and
   ↪  Action(q[restriction],[1..old.n],OnPoints)=p[restriction],"restriction pair
   ↪  mismatch");
 od;
end;;
R188Complete:=function(d)
 local colors,edges,palette,chambers,e,block,c,r,degrees,result;
 colors:=ShallowCopy(d.colors);edges:=ShallowCopy(d.edges);palette:=Set(colors);r:=Length(p
  ↪  alette);chambers:=[];
 for e in d.edges do
  block:=ShallowCopy(e);
  for c in Difference(palette,d.colors{e}) do Add(colors,c);Add(block,Length(colors));od;
  Append(edges,Combinations(block,2));Add(chambers,Set(block));
 od;
 result:=R188Data(colors,edges);degrees:=R188Degrees(result);
 R188Check(degrees{[1..d.n]}=List(R188Degrees(d),k->k*(r-1)),"old degree formula");
 R188Check(ForAll([d.n+1..result.n],v->degrees[v]=r-1),"new degree formula");
 R188Check(result.n=d.n+(r-2)*Length(d.edges),"completion size");
 return result;
end;;
R188Clean:=function(d)
 local M,W,q,colors,edges,palette,attach,base,j,k,result,deg;
 palette:=Set(d.colors);M:=Maximum(List(palette,c->Number(d.colors,x->x=c)));
 deg:=R188Degrees(d);W:=Filtered([1..d.n],v->deg[v]<=1);q:=Length(d.edges)+Length(W);
 colors:=ShallowCopy(d.colors);edges:=[];
 for attach in Concatenation(d.edges,List(W,v->[v])) do
  base:=Length(colors)+1;
```

```
  for j in [0..2*M+4] do Add(colors,Maximum(palette)+1+(j mod 2));od;
  for j in [0..2*M+3] do Add(edges,[base+j,base+j+1]);od;
  Add(edges,[base+2*M+4,base+2*M-1]);Add(edges,[base+2*M+3,base+2*M]);
  for k in attach do Add(edges,[k,base]);od;
  if Length(attach)=1 and deg[attach[1]]=0 then Add(edges,[attach[1],base+2]);fi;
 od;
 result:=R188Data(colors,edges);
 R188Check(result.n=d.n+q*(2*M+5),"cleaning size");
 R188Check(Minimum(R188Degrees(result))>=2 and R188TriangleFree(result),"cleaning graph
  ↪  hypotheses");
 return result;
end;;

# All partitions of at most three labelled vertices and every compatible graph.
clean_count188:=0;; colors:=[];;
for n in [1..3] do
 for partition in PartitionsSet([1..n]) do
  colors:=List([1..n],v->PositionProperty(partition,c->v in c));
  allowed:=Filtered(Combinations([1..n],2),e->colors[e[1]]<>colors[e[2]]);
  for edges in Combinations(allowed) do
   old188:=R188Data(colors,edges);clean188:=R188Clean(old188);
   R188CompareRestriction(old188,clean188);
   complete188:=R188Complete(clean188);R188CompareRestriction(clean188,complete188);
   clean_count188:=clean_count188+1;
  od;
 od;
od;
R188Check(clean_count188=25,"bounded graph coverage");
Print("CLEANING_AND_COMPLETION_ALL_25_COLORED_GRAPHS_PASS\n");

R188Encode:=function(G,H)
 local els,q,colors,edges,cosets,x,y,k,T,S,i,result,pair,expectedG,expectedH;
 els:=Concatenation([One(G)],Difference(Elements(G),[One(G)]));q:=Length(els);
 cosets:=Set(List(RightCosets(G,H),Set));colors:=List(els,x->PositionProperty(cosets,c->x in
  ↪  c));edges:=[];
 for x in [1..q] do for y in Difference([1..q],[x]) do
  k:=Position(els,els[y]/els[x]);T:=Length(colors)+1;Add(colors,Length(cosets)+1);
  S:=[];for i in [0..k+2] do Add(colors,Length(cosets)+2+(i mod 2));Add(S,Length(colors));od;
  Append(edges,[[x,S[1]],[S[1],y],[x,T],[T,Last(S)],[Last(S),y]]);
  Append(edges,List([1..Length(S)-1],i->[S[i],S[i+1]]));
 od;od;
 result:=R188Data(colors,edges);
 R188Check(result.n=(q^3+9*q^2-8*q)/2 and Minimum(R188Degrees(result))=2 and
  ↪  R188TriangleFree(result) and R188Connected(result),"pair encoding basic invariants");
 R188Check(Size(AutGroupGraph(R188Graph(result)))=q,"uncolored arrow rigidity");
 pair:=R188Pair(result);
 R188Check(Size(pair[1])=Size(G) and Size(pair[2])=Size(H),"normal pair orders");
 expectedG:=Group(List(GeneratorsOfGroup(G),g->PermList(List(els,x->Position(els,x*g^-1)))),
  ↪  );
 expectedH:=Group(Concatenation([()],List(GeneratorsOfGroup(H),g->PermList(List(els,x->Posi
  ↪  tion(els,x*g^-1))))));
 R188Check(Action(pair[1],[1..q],OnPoints)=expectedG and
  ↪  Action(pair[2],[1..q],OnPoints)=expectedH,"exact normal pair right action");
 if q<=3 then R188CompareRestriction(result,R188Complete(result));fi;
 Print("NORMAL_PAIR ",StructureDescription(G)," / ",StructureDescription(H)," vertices
  ↪  ",result.n," completion_checked ",q<=3," PASS\n");
end;;
for G188 in [CyclicGroup(IsPermGroup,2),CyclicGroup(IsPermGroup,3),SymmetricGroup(3)] do
 for H188 in NormalSubgroups(G188) do R188Encode(G188,H188);od;
od;

R188Family:=function(n)
 local old,colors,edges,pairs,pair,j,k,g,map,v,e,result;
 if n=2 then result:=R188Data([2,0,1,2],[[1,2],[2,3],[3,4]]);result.top:=[1,4];return
  ↪  result;fi;
 old:=R188Family(n-1);pairs:=Combinations([1..n],2);
 colors:=Concatenation(List([1..n],i->n),List(pairs,p->n-1));edges:=[];
 for j in [1..Length(pairs)] do for k in pairs[j] do Add(edges,[k,n+j]);od;od;
 for j in [1..n] do
  g:=[1..n];
  if j<>n then k:=First([1..n],i->not i in [j,n]);g[n]:=j;g[j]:=k;g[k]:=n;fi;
  map:=[];
  for k in [1..n-1] do map[old.top[k]]:=n+Position(pairs,Set([g[k],j]));od;
```

```
  for v in Difference([1..old.n],old.top) do
   ↪  Add(colors,old.colors[v]);map[v]:=Length(colors);od;
  Append(edges,List(old.edges,e->map{e}));
 od;
 result:=R188Data(colors,edges);result.top:=[1..n];return result;
end;;
family188:=rec();;
for n in [2..5] do
 family188:=R188Family(n);pair188:=R188Pair(family188);
 R188Check(Size(pair188[1])=Factorial(n) and Size(pair188[2])=Factorial(n)/2,"symmetric pair
  ↪  orders");
 R188Check(Action(pair188[1],family188.top,OnPoints)=SymmetricGroup(n) and
  ↪  Action(pair188[2],family188.top,OnPoints)=AlternatingGroup(n),"faithful natural top
  ↪  action");
 R188Check(R188Connected(family188) and R188TriangleFree(family188),"family connected
  ↪  triangle free");
 if n>=3 then R188Check(Minimum(R188Degrees(family188))>=2,"family degree");fi;
 for i in [0..n-1] do R188Check(Number(family188.colors,c->c=i)=Factorial(n)/(2*Factorial(M
  ↪  aximum(0,i-1))),"family color size");od;
 Print("SYMMETRIC_FAMILY ",n," vertices ",family188.n," color_sizes
  ↪  ",List([0..n],c->Number(family188.colors,k->k=c))," PASS\n");
od;
Print("PHASE188_BOUNDED_GADGET_REVIEW_PASS\n");
QUIT;
```

phase196_cross_ratio_check.py

```
#!/usr/bin/env python3
"""Literal finite cross-ratio incidence model; no saved tables or assertions."""
import argparse
from collections import Counter, defaultdict
from hashlib import sha256
from itertools import combinations, permutations, product
import json
from pathlib import Path


def require(ok, message):
    if not ok:
        raise RuntimeError(message)


def mul(x, y):
    z = 0
    while y:
        if y & 1:
            z ^= x
        y >>= 1
        x <<= 1
        if x & 4:
            x ^= 7
    return z


def inv(x):
    require(x != 0, 'division by zero')
    return mul(x, x)


def dot(x, y):
    z = 0
    for a, b in zip(x, y):
        z ^= mul(a, b)
    return z


def norm(v):
    c = next(x for x in v if x)
    return tuple(mul(inv(c), x) for x in v)


def cross(q, points):
```

```
    p = [points[i] for i in q]
    r, s = next((r, s) for r, s in combinations(range(3), 2)
                if mul(p[0][r], p[1][s]) ^ mul(p[0][s], p[1][r]))
    def det(i, j):
        return mul(p[i][r], p[j][s]) ^ mul(p[i][s], p[j][r])
    return mul(mul(det(0, 2), det(1, 3)), inv(mul(det(0, 3), det(1, 2))))


def main():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--output', type=Path, required=True)
    args = ap.parse_args()
    require(not args.output.exists(), 'fresh output required')
    # Check the implemented field axioms exhaustively, independently of the
    # projective computations that follow.
    require(all(mul(x, 1) == x and (x == 0 or mul(x, inv(x)) == 1)
                for x in range(4)), 'field units')
    require(all(mul(x, y) == mul(y, x) for x, y in product(range(4), repeat=2)),
            'field commutativity')
    require(all(mul(mul(x, y), z) == mul(x, mul(y, z)) and
                mul(x, y ^ z) == (mul(x, y) ^ mul(x, z))
                for x, y, z in product(range(4), repeat=3)), 'field laws')
    points = sorted({norm(v) for v in product(range(4), repeat=3) if any(v)})
    lines = [tuple(i for i, p in enumerate(points) if dot(f, p) == 0) for f in points]
    require(len(points) == len(lines) == 21 and all(len(l) == 5 for l in lines),
            'literal projective plane counts')
    require(all(sum(set(pair) <= set(l) for l in lines) == 1
                for pair in combinations(range(21), 2)), 'unique joining line')
    records = []
    fibers = defaultdict(list)
    for line, members in enumerate(lines):
        for q in permutations(members, 4):
            lam = cross(q, points)
            require(lam in (2, 3), 'nonprime cross-ratio')
            key = (lam, line, tuple(sorted(q)))
            fibers[key].append(q)
            records.append((lam, line, q))
            require(cross((q[1], q[0], q[3], q[2]), points) == lam,
                    'double interchange preserves cross-ratio')
            require(cross((q[0], q[1], q[3], q[2]), points) == inv(lam),
                    'last interchange inverts cross-ratio')
    require(Counter(r[0] for r in records) == {2: 1260, 3: 1260}, 'new type counts')
    require(len(fibers) == 210 and {len(v) for v in fibers.values()} == {12},
            'complete shadow-fiber multiplicities')
    point_frob = [points.index(norm(tuple(mul(x, x) for x in p))) for p in points]
    line_frob = [lines.index(tuple(sorted(point_frob[i] for i in l))) for l in lines]
    require(sorted(point_frob) == list(range(21)) and
            any(i != j for i, j in enumerate(point_frob)), 'nontrivial Frobenius')
    lookup = set(records)
    for lam, line, q in records:
        fq = tuple(point_frob[i] for i in q)
        target = (fq[0], fq[1], fq[3], fq[2])
        require((lam, line_frob[line], target) in lookup,
                'type-preserving Frobenius lift')
       require(tuple(point_frob[i] for i in (target[0], target[1], target[3], target[2])) ==
        ↪  q,
                'Frobenius lift squares to identity')
    key = min(fibers)
    q, q2 = fibers[key][:2]
    lam, line, support = key
    def shadow(qq):
        return {('point', i) for i in qq} | {('line', line)}
    require(q != q2 and shadow(q) == shadow(q2), 'literal distinct twin witness')
    objects = ([('point', i) for i in range(21)] +
               [('line', i) for i in range(21)] +
               [('quad', ll, ell, qq) for ll, ell, qq in records])
    u = objects.index(('quad', lam, line, q))
    v = objects.index(('quad', lam, line, q2))
    def incident(x, y):
        if x[0] == 'quad' and y[0] == 'quad':
            return x[1] != y[1]
        if y[0] == 'quad':
            x, y = y, x
        if x[0] == 'quad':
```

```
            return y[1] in x[3] if y[0] == 'point' else y[1] == x[2]
        if x[0] == y[0]:
            return False
        if x[0] == 'line':
            x, y = y, x
        return x[1] in lines[y[1]]
    def swap(i):
        return v if i == u else u if i == v else i
    def preserves(inc):
        # Pairs outside these two rows are fixed pointwise by the permutation.
        return all(inc(objects[i], objects[j]) == inc(objects[swap(i)], objects[swap(j)])
                   for i in (u, v) for j in range(len(objects)))
    require(preserves(incident), 'literal twin transposition preserves all incidences')
    def altered_incident(x, y):
        if {x, y} == {objects[v], ('point', support[0])}:
            return False
        return incident(x, y)
    require(not preserves(altered_incident), 'altered old incidence must break automorphism')
    result = {'status': 'PASS_ROOT_FINITE_CORROBORATION_UNREVIEWED',
              'points': points, 'lines': lines, 'old_type_counts': [21, 21],
              'new_type_counts': [1260, 1260], 'total_objects': 2562,
              'shadow_fibers': len(fibers), 'each_fiber_size': 12,
              'pointwise_kernel': 'Sym(12)^210',
              'witness': {'type': lam, 'line_index': line, 'support': support,
                          'first_tuple': q, 'second_tuple': q2},
              'double_interchange_checked_on': len(records),
              'frobenius_lift_checked_on': len(records),
             'negative_control': 'one removed old incidence breaks the literal transposition
              ↪  automorphism',
              'whole_source_credit': 0, 'open_conjecture_credit': 0,
              'Lean_certificate': False,
              'checker_sha256': sha256(Path(__file__).read_bytes()).hexdigest()}
    with args.output.open('x') as f:
        json.dump(result, f, indent=2)
        f.write('\n')
    print('PHASE196_FINITE_PASS', len(records), len(fibers))


if __name__ == '__main__':
    main()
```

phase197_finite_groups.g

```
LoadPackage("grape");;
SizeScreen([4096, 4096]);;
P197Need := function(ok, msg)
  if not ok then Print("P197_REJECT ", msg, "\n"); FORCE_QUIT_GAP(1); fi;
end;;
P197Need(IsBound(P197Data), "data path");;
Read(P197Data);;
P197Rows := [];;
P197Adj := [];;
for P197C in P197Cases do
  P197N := P197C.n;;
  P197R := Maximum(P197C.types);;
  P197Adj := List([1..P197N + P197R], x -> []);;
  for P197E in P197C.edges do
    AddSet(P197Adj[P197E[1]], P197E[2]);
    AddSet(P197Adj[P197E[2]], P197E[1]);
  od;
  for P197I in [1..P197N] do
    P197J := P197N + P197C.types[P197I];;
    AddSet(P197Adj[P197I], P197J);
    AddSet(P197Adj[P197J], P197I);
  od;
  P197Graph := Graph(Group(()), [1..P197N + P197R], OnPoints,
    function(x, y) return y in P197Adj[x]; end, true);;
  P197Old := [1..P197N];;
  P197Palette := [P197N + 1..P197N + P197R];;
  P197Corr := AutGroupGraph(P197Graph, [P197Old, P197Palette]);;
  P197Typed := AutGroupGraph(P197Graph,
    Concatenation([P197Old], List(P197Palette, x -> [x])));;
```

```
  P197Need([Size(P197Typed), Size(P197Corr)] = P197C.expected,
    Concatenation("group orders ", P197C.name));
  P197Image := Action(P197Corr, P197Palette);;
  P197Need(Size(P197Corr) = Size(P197Typed) * Size(P197Image), "type quotient");
  for P197G in GeneratorsOfGroup(P197Corr) do
    for P197I in P197Old do
      P197Need(P197C.types[P197I ^ P197G] + P197N =
        (P197C.types[P197I] + P197N) ^ P197G, "exact palette action");
    od;
  od;
  Add(P197Rows, Concatenation("{\"name\":\"", P197C.name,
    "\",\"typed\":", String(Size(P197Typed)),
    ",\"correlations\":", String(Size(P197Corr)),
    ",\"type_image\":", String(Size(P197Image)), "}"));
od;
Print("{\"status\":\"PASS_NATIVE_GAP_FULL_GROUPS\",\"rows\":[",
  JoinStringsWithSeparator(P197Rows, ","), "]}\n");;
QUIT;
```

phase197_finite_models.py

```
#!/usr/bin/env python3
"""Literal and explicitly repaired L126 models. No assertion-dependent checks."""
import argparse
import itertools as it
import json
import math
from pathlib import Path


def need(condition, message):
    if not condition:
        raise ValueError("P197_REJECT " + message)


def parity(p):
    return sum(p[i] > p[j] for i in range(len(p)) for j in range(i + 1, len(p))) % 2


def model(name, objects, types, incidence, expected, geometric=None):
    need(len(objects) == len(types), name + " type length")
    n = len(objects)
    edges = [[i + 1, j + 1] for i in range(n) for j in range(i + 1, n)
             if incidence(i, j)]
    need(all(types[i - 1] != types[j - 1] for i, j in edges), name + " same-type incidence")
    neighbors = [set() for _ in objects]
    for i, j in edges:
        neighbors[i - 1].add(j - 1)
        neighbors[j - 1].add(i - 1)
    rank = len(set(types))
    flag_counts = [0] * (rank + 1)
    nonextendable = []

    def flags(flag, available):
        flag_counts[len(flag)] += 1
        if flag and len(flag) < rank and not set.intersection(*(neighbors[x] for x in flag)):
            nonextendable.append([x + 1 for x in flag])
        for x in sorted(available):
            flags(flag + [x], {y for y in available & neighbors[x] if y > x})

    flags([], set(range(n)))
    is_geometry = not nonextendable and flag_counts[-1] > 0
    if geometric is not None:
        need(is_geometry == geometric, name + " geometry status")
    return dict(name=name, n=n, rank=rank, types=types, edges=edges,
                expected_groups=expected, flag_counts=flag_counts,
                geometry=is_geometry, nonextendable_witness=next(iter(nonextendable), None))


def dihedral(n):
    directions = [k for k in range(1, (n + 1) // 2) if math.gcd(k, n) == 1]
    obj = [("p", i) for i in range(n)]
```

```
    typ = [0 if n % 2 else i % 2 for i in range(n)]
    for k in directions:
        obj += [("e", frozenset((x, (x + k) % n))) for x in range(n)]
        typ += [len(set(typ))] * n

    def inc(i, j):
        a, b = obj[i], obj[j]
        if a[0] == b[0] == "p":
            return n % 2 == 0 and typ[i] != typ[j]
        if a[0] == b[0] == "e":
            return typ[i] != typ[j]
        return a[1] in b[1]

    phi = sum(math.gcd(k, n) == 1 for k in range(n))
    return model("dihedral_" + str(n), obj, typ, inc,
                 [6, 12] if n == 3 else [2 * n if n % 2 else n, n * phi])


def symmetric(n):
    obj = [("p", i) for i in range(n)] + [("e", frozenset(e)) for e in
     ↪  it.combinations(range(n), 2)]
    typ = [0] * n + [1] * (len(obj) - n)
    return model("complete_" + str(n), obj, typ,
                lambda i, j: obj[i][0] == "p" and obj[j][0] == "e" and obj[i][1] in obj[j][1],
                 [math.factorial(n), 12 if n == 3 else math.factorial(n)], True)


def cube(repaired):
    vertices = list(it.product(range(2), repeat=3))
    edges = [frozenset(e) for e in it.combinations(range(8), 2)
             if sum(a != b for a, b in zip(vertices[e[0]], vertices[e[1]])) == 1]
    faces = [frozenset(i for i, v in enumerate(vertices) if v[k] == bit)
             for k in range(3) for bit in range(2)]
    supports = [frozenset([i]) for i in range(8)] + edges + faces
    typ = [sum(v) % 2 for v in vertices] + [2] * 12 + [3] * 6

    def inc(i, j):
        if typ[i] == typ[j]:
            return False
        if i < 8 and j < 8:
            return repaired and frozenset((i, j)) in edges
        return supports[i] <= supports[j] or supports[j] <= supports[i]

    return model("cube_repaired" if repaired else "cube_literal", supports, typ, inc, [24,
     ↪  48], repaired)


def petersen():
    vertices = list(map(frozenset, it.combinations(range(5), 2)))
    edges = [frozenset((i, j)) for i, j in it.combinations(range(10), 2) if not vertices[i] &
     ↪  vertices[j]]
    cycle_data = []
    for tail in it.permutations(range(1, 5)):
        order = (0,) + tail
        if order[1] > order[-1]:
            continue
        support = frozenset(vertices.index(frozenset((order[i], order[(i + 1) % 5]))) for i
         ↪  in range(5))
        cycle_edges = frozenset(e for e in edges if e <= support)
        need(len(cycle_edges) == 5, "Petersen pentagon")
        cycle_data.append((parity(order), order, support, cycle_edges))
    cycle_data.sort()
    need([x[0] for x in cycle_data] == [0] * 6 + [1] * 6, "Petersen halves")
    supports = [frozenset([i]) for i in range(10)] + edges + [x[2] for x in cycle_data]
    typ = [0] * 10 + [1] * 15 + [2 + x[0] for x in cycle_data]
    hist = {}
    for a in cycle_data[:6]:
        for b in cycle_data[6:]:
            shared_edges = a[3] & b[3]
            shared_points = a[2] & b[2]
            key = (len(shared_points), len(shared_edges))
            hist[str(key)] = hist.get(str(key), 0) + 1
            need(set().union(*shared_edges) == set(shared_points), "common vertices lie on
             ↪  common edges")
```

```
    need(hist == {"(3, 2)": 30, "(0, 0)": 6}, "Petersen cross-cycle table")

    def inc(i, j):
        if typ[i] == typ[j]:
            return False
        if i >= 25 and j >= 25:
            return bool(cycle_data[i - 25][3] & cycle_data[j - 25][3])
        return supports[i] <= supports[j] or supports[j] <= supports[i]

    out = model("petersen_completed", supports, typ, inc, [60, 120], True)
    out["cross_cycle_intersections"] = hist
    out["cycles"] = [dict(parity=a, order=b, support=sorted(c), edges=[sorted(e) for e in
     ↪  sorted(d, key=lambda x: sorted(x))])
                     for a, b, c, d in cycle_data]
    need(out["flag_counts"][-1] == 120, "Petersen chamber count")
    return out


def doily():
    duads = list(map(frozenset, it.combinations(range(6), 2)))
    triples = [frozenset(c) for c in it.combinations(range(15), 3)
               if len(set().union(*(duads[i] for i in c))) == 6]
    need(len(triples) == 15, "syntheme count")
    return model("duad_syntheme", duads + triples, [0] * 15 + [1] * 15,
                 lambda i, j: i < 15 <= j and i in triples[j - 15], [720, 1440], True)


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--gap-output", type=Path, required=True)
    parser.add_argument("--alter-petersen", action="store_true")
    args = parser.parse_args()
    cases = [dihedral(n) for n in range(3, 13)] + [symmetric(n) for n in range(2, 9)]
    cases += [cube(False), cube(True), petersen(), doily()]
    if args.alter_petersen:
        target = next(c for c in cases if c["name"] == "petersen_completed")
        removed = next(e for e in target["edges"] if min(e) >= 26)
        target["edges"].remove(removed)
        need(target["edges"] == petersen()["edges"], "altered Petersen cross-incidence")
    gap = ["P197Cases := ["]
    for c in cases:
        gap.append("rec(name:=" + json.dumps(c["name"]) + ", n:=" + str(c["n"]) +
                   ", types:=" + str([t + 1 for t in c["types"]]) + ", edges:=" +
                    ↪  str(c["edges"]) +
                   ", expected:=" + str(c["expected_groups"]) + "),")
    gap[-1] = gap[-1].rstrip(",")
    gap += ["];;", ""]
    args.output.write_text(json.dumps(dict(status="PASS_FINITE_LITERAL_AND_EXPLICIT_COMPLET
     ↪  IONS", cases=cases), indent=2) + "\n")
    args.gap_output.write_text("\n".join(gap))


if __name__ == "__main__":
    main()
```

phase197_free_basis_check.py

```
#!/usr/bin/env python3
"""Bounded checks of the reduced-word criterion and finite source core graphs."""
import argparse
import itertools as it
import json
from pathlib import Path


def need(ok, message):
    if not ok:
        raise ValueError("P197_REJECT " + message)


def words(rank, length):
```

```
    current = [()]
    yield ()
    alphabet = [s * i for i in range(1, rank + 1) for s in (1, -1)]
    for _ in range(length):
        current = [w + (a,) for w in current for a in alphabet if not w or w[-1] != -a]
        yield from current


def factor_membership(w, omitted, i):
    # Independent of the last-i prefix: try every cut in the reduced word.
    return any(not (set(map(abs, w[:k])) & omitted) and i not in map(abs, w[k:])
               for k in range(len(w) + 1))


def check_words(rank, length, alter_prefix):
    total = 0
    for w in words(rank, length):
        support = set(map(abs, w))
        for i in range(1, rank + 1):
            occurrences = [k + 1 for k, a in enumerate(w) if abs(a) == i]
            end = (min if alter_prefix else max)(occurrences, default=0)
            prefix_support = set(map(abs, w[:end]))
            others = [j for j in range(1, rank + 1) if j != i]
            for size in range(rank):
                for js in it.combinations(others, size):
                    omitted = set(js)
                    cut = factor_membership(w, omitted, i)
                    need(cut == (not omitted & prefix_support), "last-occurrence cut
                     ↪  criterion")
                    pairwise = all(factor_membership(w, {j}, i) for j in js)
                    need(cut == pairwise, "all-factor product identity")
        total += 1
    return dict(rank=rank, max_length=length, reduced_words=total)


def core(n):
    # Positive labels orient edges; record outgoing positive and inverse labels.
    oriented = {0: set()}
    edges = []
    for j in range(1, n + 1):
        for sign in (1, -1):
            v = (j, sign)
            oriented[v] = set()
            edges.append((0, v, j * sign))
            for i in range(1, n + 1):
                if i != j:
                    edges.append((v, v, i))
    for u, v, label in edges:
        need(label not in oriented[u], "folded graph outgoing label")
        oriented[u].add(label)
        need(-label not in oriented[v], "folded graph inverse label")
        oriented[v].add(-label)
    rank = len(edges) - len(oriented) + 1
    need(rank == 2 * n * (n - 1), "core rank")
    return dict(ambient_rank=n, vertices=len(oriented), unoriented_edges=len(edges),
     ↪  free_rank=rank)


def signed_type_actions(n):
    basis = [(j, sign, i) for j in range(n) for sign in (1, -1) for i in range(n) if i != j]
    actions = set()
    signed_actions = set()
    for p in it.permutations(range(n)):
        for eps in it.product((1, -1), repeat=n):
            action = tuple(basis.index((p[j], sign * eps[j], p[i])) for j, sign, i in basis)
            signed = tuple((action[k], eps[i]) for k, (_, _, i) in enumerate(basis))
            actions.add(action)
            signed_actions.add(signed)
    need(len(actions) == len(signed_actions) == 2**n * __import__('math').factorial(n),
     ↪  "ambient signed subgroup")
    if n == 2:
        three_cycle = (1, 2, 0, 3)
        need(three_cycle not in actions, "basis three-cycle outside source square group")
```

```
        need(len(set(it.permutations(range(4)))) == 24 and len(actions) == 8, "S4 versus
         ↪  square group")
    return dict(ambient_rank=n, basis_size=len(basis), source_type_image_order=len(actions))


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--output", type=Path, required=True)
    p.add_argument("--alter-prefix", action="store_true")
    args = p.parse_args()
    result = dict(status="PASS_BOUNDED_FREE_WORD_AND_CORE_CHECKS",
                  words=[check_words(3, 6, args.alter_prefix), check_words(4, 5,
                   ↪  args.alter_prefix)],
                  cores=[core(n) for n in range(2, 7)],
                  signed_actions=[signed_type_actions(n) for n in range(2, 5)])
    args.output.write_text(json.dumps(result, indent=2) + "\n")


if __name__ == "__main__":
    main()
```

ordered_arms.g

```
# Optional finite diagnostics for the uniform rigid-arm theorem.
# Full proof is written; these three small tuples are not a general certificate.
if LoadPackage("grape")=fail then Error("GRAPE unavailable");fi;
Need:=function(b,s) if not b then Error(Concatenation("L126_ARMS_REJECT ",s));fi;end;
Arms:=function(d,split)
 local tuples,colors,edges,t,center,i,k,last,v,n,cls,palette,aug,C,K,base,
       gr,A,cor,kernel,p,res,seen,perm,expected;
 tuples:=PermutationsList([1..d]);colors:=List([1..d],x->1);edges:=[];
 for t in tuples do
  center:=Length(colors)+1;
  if split then Add(colors,2+(1-SignPerm(PermList(t)))/2);else Add(colors,2);fi;
  for i in [1..d] do
   last:=center;
   for k in [1..i] do
    v:=Length(colors)+1;Add(colors,10+10*i+k);
    Add(edges,Set([last,v]));last:=v;
   od;
   Add(edges,Set([last,t[i]]));
  od;
 od;
 n:=Length(colors);edges:=Set(edges);palette:=Set(colors);
 cls:=List(palette,c->Filtered([1..n],i->colors[i]=c));
 gr:=Graph(Group(()),[1..n],OnPoints,function(a,b)return Set([a,b]) in edges;end,true);
 aug:=Graph(Group(()),[1..n+Length(palette)],OnPoints,function(a,b)
  if a<=n and b<=n then return Set([a,b]) in edges;fi;
  if a>n and b>n then return false;fi;
  if a<=n then return colors[a]=palette[b-n];fi;
  return colors[b]=palette[a-n];
 end,true);
 A:=AutGroupGraph(aug,[[1..n],[n+1..n+Length(palette)]]);
 C:=Action(A,[1..n],OnPoints);K:=AutGroupGraph(gr,cls);
 Need(Size(A)=Size(C),"faithful palette encoding");
 Need(ForAll(GeneratorsOfGroup(C),g->OnSets([1..d],g)=[1..d]),"recognizable old objects");
 res:=Action(C,[1..d],OnPoints);
 Need(res=SymmetricGroup(d),"full restriction action");
 kernel:=Stabilizer(C,[1..d],OnTuples);Need(Size(kernel)=1,"ordered rigidity");
 expected:=Factorial(d);if split then expected:=expected/2;fi;
 Need(Size(K)=expected and Size(C)=Factorial(d),"complete group pair");
 Print("ARMS_PASS d=",d," split=",split," vertices=",n," C=",Size(C)," K=",Size(K),"\n");
end;
Arms(3,false);Arms(4,false);Arms(4,true);
Print("L126_ORDERED_ARMS_COMPLETE\n");QUIT;
```

Required elementary graph certificate: `certificate/check.py`

```
#!/usr/bin/env python3
"""Elementary exhaustive certificate; no graph/group package is imported."""
```

```
import argparse
import collections
import hashlib
import json
import math
from pathlib import Path


class Rejected(ValueError):
    pass


def require(condition, message):
    if not condition:
        raise Rejected(message)


def refinement(adj, star):
    colors = [0] * len(adj)
    for i, v in enumerate(star):
        colors[v] = i + 1
    counts = [len(set(colors))]
    while True:
        signatures = [(colors[v], tuple(sorted(colors[w] for w in adj[v])))
                      for v in range(len(adj))]
        dictionary = {s: i for i, s in enumerate(sorted(set(signatures)))}
        new = [dictionary[s] for s in signatures]
        counts.append(len(dictionary))
        if len(dictionary) == len(set(colors)):
            return new, counts
        colors = new


def group_closure(generators, n, upper):
    identity = tuple(range(n))
    seen = {identity}
    todo = collections.deque([identity])
    while todo:
        p = todo.popleft()
        for g in generators:
            q = tuple(g[p[v]] for v in range(n))
            if q not in seen:
                seen.add(q)
                require(len(seen) <= upper, 'group exceeds certified upper bound')
                todo.append(q)
    return seen


def permutation_order(p):
    seen = set()
    order = 1
    for v in range(len(p)):
        if v in seen:
            continue
        w, length = v, 0
        while w not in seen:
            seen.add(w)
            length += 1
            w = p[w]
        order = math.lcm(order, length)
    return order


def inspect(data, expected=None, require_nonsplit=True):
    adj = [tuple(v - 1 for v in row) for row in data['adjacency']]
    n = len(adj)
    vertices = set(range(n))
    degree = len(adj[0])
    require(degree >= 2, 'minimum degree')
    for v, row in enumerate(adj):
        require(len(row) == degree and len(set(row)) == degree, 'simple regular graph')
        require(v not in row and set(row) <= vertices, 'valid loop-free adjacency')
        require(all(v in adj[w] for w in row), 'symmetric adjacency')
    sides = {0: 0}
```

```
    todo = collections.deque([0])
    while todo:
        v = todo.popleft()
        for w in adj[v]:
            if w not in sides:
                sides[w] = 1 - sides[v]
                todo.append(w)
            require(sides[w] != sides[v], 'bipartite')
    require(len(sides) == n, 'connected')
    edges = {tuple(sorted((v, w))) for v in vertices for w in adj[v]}
    part = {v for v in vertices if sides[v] == 0}
    require(len(part) * 2 == n and len(part) != len(edges), 'part and edge sizes')
    star = [v - 1 for v in data['distinguished_star']]
    require(len(star) == degree + 1 and set(star[1:]) == set(adj[star[0]]), 'full star')
    colors, counts = refinement(adj, star)
    require(len(set(colors)) == n, 'star refinement is not discrete')
    # Every automorphism is determined by its images on this ordered star.
    upper = n * math.factorial(degree)
    generators = [tuple(v - 1 for v in row) for row in data['generators']]
    for g in generators:
        require(len(g) == n and set(g) == vertices, 'generator is not permutation')
        require(all({g[w] for w in adj[v]} == set(adj[g[v]]) for v in vertices),
                'generator does not preserve graph')
    group = group_closure(generators, n, upper)
    require(len(group) == upper, 'generated group does not attain full upper bound')
    inside, outside = [], []
    for g in group:
        image = {g[v] for v in part}
        require(image == part or image == vertices - part, 'part transport')
        (inside if image == part else outside).append(g)
    require(len(inside) == len(outside), 'part action is not onto C2')
    edge = min(edges)
    edge_orbit = {tuple(sorted((g[edge[0]], g[edge[1]]))) for g in inside}
    require(edge_orbit == edges, 'part-preserving subgroup is not edge-transitive')
    orders = dict(sorted(collections.Counter(permutation_order(g) for g in outside).items()))
    if require_nonsplit:
        require(1 not in orders and 2 not in orders, 'part-exchanging involution')
    result = {
        'vertices': n, 'edges': len(edges), 'degree': degree,
        'part_sizes': [len(part), n - len(part)],
        'refinement_cell_counts': counts,
        'full_automorphism_upper_bound': upper,
        'generated_group_order': len(group),
        'generator_orders': [permutation_order(g) for g in generators],
        'part_preserving_order': len(inside),
        'part_preserving_edge_orbit_size': len(edge_orbit),
        'exchanging_coset_order_distribution': orders,
        'fullness': 'discrete star refinement upper bound attained by explicit subgroup',
    }
    if expected is not None:
        require(result['vertices'] == expected['vertices'], 'expected vertex count')
        require(result['edges'] == expected['edges'], 'expected edge count')
        require(result['generated_group_order'] == expected['group_order'], 'expected full
         ↪  order')
        require(orders == expected['orders'], 'expected exchanging order distribution')
    return result


def controls(data):
    import copy
    cases = []
    def rejection(name, changed, intended):
        try:
            inspect(changed)
        except Rejected as error:
            require(intended in str(error), 'control rejected at wrong interface: ' + name)
            cases.append({'name': name, 'rejected_at': str(error)})
        else:
            raise Rejected('negative control was accepted: ' + name)
    d = copy.deepcopy(data)
    d['adjacency'][0][0] = 1
    rejection('loop in adjacency', d, 'loop-free')
    d = copy.deepcopy(data)
    d['generators'][0][0] = d['generators'][0][1]
```

```
    rejection('nonbijective generator', d, 'not permutation')
    d = copy.deepcopy(data)
    g = list(range(1, len(d['adjacency']) + 1))
    g[0], g[1] = g[1], g[0]
    d['generators'] = [g]
    rejection('nongraph permutation', d, 'does not preserve graph')
    d = copy.deepcopy(data)
    d['generators'] = [list(range(1, len(d['adjacency']) + 1))]
    rejection('proper subgroup substituted for full group', d, 'does not attain')
    d = copy.deepcopy(data)
    d['distinguished_star'] = [1]
    rejection('incomplete star bound', d, 'full star')
    # This connected bipartite edge-transitive graph passes every preceding
    # general interface, including the elementary full-group certificate.
    d = {'adjacency': [[(v - 1) % 6 + 1, (v + 1) % 6 + 1] for v in range(6)],
         'generators': [[(v + 1) % 6 + 1 for v in range(6)],
                        [(-v) % 6 + 1 for v in range(6)]],
         'distinguished_star': [1, 6, 2]}
    rejection('C6 swapping involutions', d, 'part-exchanging involution')
    # Even the actual full star must be distinguished pointwise for the upper bound.
    adj = [tuple(v - 1 for v in row) for row in data['adjacency']]
    colors, _ = refinement(adj, [0])
    require(len(set(colors)) < len(adj), 'root-only weakening unexpectedly discrete')
    cases.append({'name': 'root-only refinement weakening', 'rejected_at': 'nondiscrete
    ↪  partition'})
    return cases


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--data', type=Path, default=Path(__file__).with_name('graph.json'))
    parser.add_argument('--out', type=Path, required=True)
    args = parser.parse_args()
    raw = args.data.read_bytes()
    data = json.loads(raw)
    result = inspect(data, {'vertices': 896, 'edges': 1344, 'group_order': 5376,
                            'orders': {4: 1024, 12: 896, 28: 768}})
    result['controls'] = controls(data)
    result['data_sha256'] = hashlib.sha256(raw).hexdigest()
    result['checker_sha256'] = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
    result['status'] = 'PASS'
    args.out.parent.mkdir(parents=True, exist_ok=True)
    args.out.write_text(json.dumps(result, indent=2) + '\n')
    print(json.dumps(result, sort_keys=True))


if __name__ == '__main__':
    main()
```

Optional full-group diagnostic: `certificate/diagnostic.py`

```
#!/usr/bin/env python3
"""Optional two-backend validation, not a premise of the elementary proof."""
import argparse
import hashlib
import json
import subprocess
from pathlib import Path

p = argparse.ArgumentParser()
p.add_argument('--gap', default='/opt/homebrew/bin/gap')
p.add_argument('--out', type=Path, required=True)
a = p.parse_args()
data_path = Path(__file__).with_name('graph.json')
data = json.loads(data_path.read_bytes())
script = '''
if LoadPackage("grape")=fail or LoadPackage("digraphs")=fail then Error("packages");fi;
adj:=%s;;gens:=%s;;
B:=Graph(Group(()),[1..Length(adj)],OnPoints,function(u,v)return v in adj[u];end,true);;
A:=AutGroupGraph(B);;
D:=Digraph(adj);;C:=BlissAutomorphismGroup(D);;
P:=Group(List(gens,PermList));;
```

```
if A<>C or A<>P or Size(A)<>5376 then Error("full group discrepancy");fi;
part:=Filtered([1..896],v->Distance(B,1,v) mod 2=0);;
K:=Stabilizer(A,Set(part),OnSets);;
edges:=Set(Concatenation(List([1..896],u->List(Filtered(adj[u],v->v>u),v->[u,v]))));;
if Size(K)<>2688 or Length(Orbits(K,edges,OnSets))<>1 then Error("part action");fi;
distribution:=Collected(List(Filtered(Elements(A),g->not g in K),Order));;
if distribution<>[[4,1024],[12,896],[28,768]] then Error("coset distribution");fi;
Print("GAP ",GAPInfo.Version," grape ",PackageInfo("grape")[1].Version,
 " digraphs ",PackageInfo("digraphs")[1].Version,"\\n");
Print("L126_TWO_BACKEND_PASS full=5376 kernel=2688 orders=",distribution,"\\n");
QUIT;
''' % (repr(data['adjacency']), repr(data['generators']))
r = subprocess.run([a.gap, '-q', '-b'], input=script, text=True,
                   capture_output=True, timeout=120)
if r.returncode != 0 or 'L126_TWO_BACKEND_PASS' not in r.stdout or 'Error,' in r.stdout +
 ↪  r.stderr:
    raise SystemExit('GAP diagnostic failed\n' + r.stdout + r.stderr)
result = {'status': 'PASS', 'role': 'optional validation only',
          'data_sha256': hashlib.sha256(data_path.read_bytes()).hexdigest(),
          'diagnostic_sha256': hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
          'stdout': r.stdout, 'stderr': r.stderr}
a.out.parent.mkdir(parents=True, exist_ok=True)
a.out.write_text(json.dumps(result, indent=2) + '\n')
print(r.stdout.strip())
```